

Cuaderno de ejercicios

Resuelve en un laboratorio aislado. Las soluciones están en un documento separado. Para cada prueba registra observación, hipótesis, resultado esperado, resultado obtenido y conclusión.

Nivel 1: identificar y leer

E01 - Entradas HTTP

En la petición siguiente enumera todas las entradas controlables y el parser probable de cada una:

```
POST /api/avatar?size=80 HTTP/1.1
Host: perfil.lab
Cookie: session=s-123
Content-Type: application/json

{"url":"http://img.lab/a.png","crop":{"x":0,"y":0}}
```

Indica cuáles podrían seleccionar recurso, identidad, destino de red y parámetros numéricos.

E02 - Source y sink

```
name = request.args["name"]
safe = name.replace(";", "")
result = subprocess.run("lookup " + safe, shell=True, capture_output=True)
```

Marca source, transformación, validación, intérprete y sink. Explica qué propiedad intenta garantizar el reemplazo y por qué esa propiedad es insuficiente.

E03 - Leer una respuesta

Una petición normal devuelve 200, 4.120 bytes y 85 ms. Una comilla devuelve 500, 780 bytes y 82 ms. Una segunda comilla también devuelve 500. Clasifica el hallazgo y diseña un control que permita distinguir un error SQL de otro error de aplicación.

E04 - Puerto y servicio

Un escaneo muestra TCP/8080 abierto. Redacta tres hipótesis y una prueba mínima para cada una. No uses el nombre de puerto como confirmación.

Nivel 2: construir pruebas

E05 - SQL por contexto

Compara:

```
SELECT * FROM items WHERE id = INPUT
SELECT * FROM items WHERE name = 'INPUT'
```

Explica por qué la misma comilla no tiene la misma función. Diseña pares verdadero/falso sin extraer datos.

E06 - Command o argument injection

Escenario A ejecuta `sh -c "ping -c 1 " + host`. Escenario B ejecuta `execve("ping", ["ping", "-c", "1", host])`. Predice el significado de `127.0.0.1; id` en ambos. Diseña una prueba inocua para distinguirlos.

E07 - Traversal

La aplicación abre `/srv/reports/` más el valor `file`. El error revela `/srv/reports/2026/julio/no.txt`. Construye la ruta relativa mínima para solicitar `/srv/marker.txt` y muestra la normalización paso a paso.

E08 - SSRF con callback

Hay un parámetro `url`. Diseña una prueba HTTP hacia un listener propio, un control negativo y la evidencia mínima. Indica qué verificarías antes de probar loopback.

E09 - Upload por capas

Diseña una matriz de pruebas que cambie una sola variable: nombre, Content-Type, bytes, extensión almacenada y ruta de servicio. El objetivo es saber qué capa decide la aceptación y cuál la interpretación.

E10 - SSTI

La cadena `{{7*7}}` aparece como 49; `{{7*'7'}}` aparece como siete repeticiones. Explica qué está confirmado y qué no. Indica el siguiente paso basado en identificación del motor, no en una cadena RCE copiada.

Nivel 3: adaptar y descartar

E11 - Normalización

Un filtro rechaza la cadena literal `../`; el backend registra que `%2e%2e%2f` llega como `../`. Dibuja el orden de validación y decodificación. Propón una defensa correcta y una prueba para verificarla.

E12 - Falso positivo temporal

Cinco peticiones de control tardan 100, 140, 900, 120 y 110 ms. Una prueba temporal tarda 820 ms una vez. ¿Qué conclusión es válida? Diseña una medición mejor.

E13 - Autorización

Dos usuarios pueden solicitar `/api/invoices/41`. Usuario A obtiene 200; usuario B obtiene 403. Al cambiar a `/api/invoices/42`, B obtiene 200. Explica qué debes comprobar antes de afirmar IDOR y qué evidencia confirmaría acceso cruzado.

E14 - sudo

`sudo -l` permite `/usr/bin/find` como root sin contraseña. Sin ejecutar una shell, describe cómo verificarías la regla, versión, comportamiento relevante e identidad efectiva. Explica por qué una receta de otro binario no es transferible.

E15 - Servicio Windows

`sc qc DemoSvc` muestra `C:\Program Files\Demo App\service.exe` sin comillas y cuenta LocalSystem. ¿Es suficiente para confirmar escalada? Enumera permisos y condiciones adicionales necesarios.

E16 - Kerberos

Un ccache válido falla con `KRB_AP_ERR_SKEW`. Formula hipótesis, prueba mínima y adaptación. Distingue autenticación fallida por hora de autorización denegada.

Nivel 4: casos completos

E17 - Burp Repeater

Una API acepta JSON y devuelve campos diferentes según rol. Diseña una secuencia de seis peticiones que estudie autenticación, autorización de objeto, método, campo client-side y sesión, cambiando una variable cada vez.

E18 - Terminal Linux

Tienes shell como `www-data`. Construye un plan de enumeración basado en evidencia para `sudo`, `SUID`, `capabilities`, `cron`, grupos y secretos. Para cada categoría indica salida que haría avanzar y salida que la descartaría.

E19 - Room tipo TryHackMe

Observas 22/SSH, 80/HTTP y 445/SMB. La web redirige a un nombre que no resuelve; SMB permite sesión nula pero no lista shares. Prioriza diez pruebas y justifica el orden. Incluye criterios para abandonar cada hipótesis.

E20 - Encadenamiento y falso positivo

Una LFI lee un archivo de configuración con una contraseña; esa contraseña funciona en SSH para un usuario sin `sudo`. Un `cron` de root ejecuta un script legible pero no escribible en un directorio escribible. Construye la cadena de hipótesis. Identifica qué es confirmación, qué es solo indicio y qué permisos debes verificar antes de afirmar impacto.

Nivel 5: razonamiento y transferencia

En E21-E27 entrega siempre esta cadena: observación → qué sabes realmente → hipótesis rivales → experimento discriminatorio → resultados esperados → resultado simulado → conclusión → siguiente paso. No puntúa un payload sin reconstrucción del parser y control negativo.

E21 - Qué probarías ahora

Un conversor de imágenes recibe `source=https://img.lab/a.png`. La respuesta tarda 180 ms y contiene `converted=true`, pero tu navegador también solicita esa URL al mostrar la vista previa. Dispones de un listener con tokens únicos.

Diseña exactamente las tres siguientes peticiones. Deben distinguir petición del navegador, petición del servidor, caché y validación sin conexión. Define qué observarías en navegador, listener y respuesta para cada hipótesis.

E22 - Construcción de payload por capas

Una API recibe JSON:

```
{"filter":{"name = 'paper'}}
```

El valor se inserta en `SELECT id,name FROM stock WHERE <filter> AND active=1`. El laboratorio usa un motor aún desconocido. Construye una pareja verdadera/falsa que conserve la consulta válida sin extraer datos.

Descompón contexto, sintaxis, entrada controlada, parser, sink, primitiva y significado de cada componente. Incluye petición original y filtro inexistente como controles.

E23 - Diagnóstico de payload fallido

Una función de diagnóstico acepta `host.127.0.0.1; printf TOKEN` devuelve `unknown host`, con `TOKEN` incluido literalmente en el mensaje. Una demora añadida no cambia la distribución temporal. El equipo afirma que “el punto y coma está filtrado”.

Propón al menos tres hipótesis rivales, entre ellas shell ausente, quoting y validación previa. Diseña una prueba inocua por hipótesis y explica por qué cambiar de separador todavía no está justificado.

E24 - Hipótesis rivales

Después de subir `avatar.svg`, la aplicación responde 201 y `/media/7f31`. Al abrir la ruta, devuelve `Content-Type: application/octet-stream`, `Content-Disposition: attachment` y los bytes originales. En otra cuenta, `/media/7f31` devuelve 403.

Separa las hipótesis: subida arbitraria, almacenamiento, lectura pública, interpretación en navegador, ejecución server-side y autorización correcta. Diseña el experimento mínimo que confirma o descarta cada una sin introducir código ejecutable.

E25 - Reconocimiento del parser

Compara dos implementaciones no visibles:

```
run(["fetch", user_value])
run(["fetch", *split(user_value)])
```

Con `user_value = "https://a.lab/x https://b.lab/y"`, el sistema registra dos callbacks pero ningún metacarácter de shell produce efecto. Diseña pruebas con comillas, espacios escapados y un operando que empiece por `-` para inferir: número de elementos de `argv`, reglas de splitting y parser de opciones. No uses ejecución de comandos como criterio único.

E26 - El importador nocturno

Una aplicación permite registrar una URL de catálogo. La interfaz solo muestra “importación programada”. Horas después, tu listener recibe `GET /catalog/42` con `User-Agent: curl/8.x`; una URL que redirige a otra ruta genera un segundo callback. Un nombre que alterna entre una IP pública y una privada produce resultados inconsistentes.

Sin nombrar la vulnerabilidad, reconstruye componentes y límites de confianza. Diseña experimentos para separar validación al guardar, resolución al ejecutar, seguimiento de redirecciones, caché DNS y política sobre la dirección efectiva. El objetivo no es alcanzar una red interna, sino demostrar en el laboratorio dónde debe aplicarse cada

control.

E27 - Mini-room: el informe de medianoche

Dispones de una aplicación de informes y una shell inicial como reporter. Observas:

- `POST /render` acepta una plantilla y devuelve un PDF.
- Una entrada especial causa error que muestra
`argv=["wkhtmltopdf","--quiet","/tmp/job-81.html","/tmp/job-81.pdf"]`.
- `sudo -l` permite como `archive` exactamente `/usr/local/bin/archive /srv/reports/daily`.
- Un timer ejecuta como `root /opt/cleanup/run.sh`; el script es propiedad de `root` y no escribible.
- El script llama `rotate-report` sin ruta absoluta; la traza del timer muestra `PATH=/opt/cleanup/bin:/usr/bin`.
- `/opt/cleanup/bin` pertenece a `root:reports` y tiene modo `0775`; `reporter` pertenece a `reports`.

Construye una ruta de investigación de máximo ocho experimentos. Incluye controles negativos, identidad efectiva, argumentos permitidos, `PATH` del consumidor y una prueba de impacto mínimo mediante marcador. Señala dos falsos positivos plausibles y el criterio exacto para descartarlos. No asumas que el error de `argv`, la regla `sudo` o el script legible equivalen por sí solos a ejecución privilegiada.

Preguntas tipo test

1. ¿Qué confirma mejor una SQLi booleana? A) un 500; B) un par verdadero/falso reproducible; C) una página lenta; D) una versión en cabecera.
2. ¿Qué capa suele consumir `#fragment` de una URL HTTP? A) servidor; B) DNS; C) cliente; D) TLS.
3. En `execve` con `argv` fijo, ; es normalmente: A) separador shell; B) dato dentro de un argumento; C) pipe; D) redirección.
4. Una arista `BloodHound` es: A) impacto confirmado; B) ruta potencial que requiere precondiciones; C) contraseña; D) ticket.
5. La mejor defensa ante `traversal` es: A) quitar `../`; B) doble URL decode; C) IDs indirectos y ruta canónica confinada; D) bloquear puntos.